Theresa Dryden
November 11, 2026
Introduction to Programming with Python
Assignment 05 – Module 05: Advanced Collections and Error Handling

Introduction

The purpose of Assignment 05 is to extend the course registration program developed in Assignment 04. The earlier version used a two-dimensional list and a CSV file; this version uses a list of dictionaries, stores registrations in JSON, and introduces structured exception handling. The program continues to register students, display current enrollments, save data, and exit through a four-option menu.

Planning and Reviewing the Starter Script

I reviewed Assignment05-Starter.py and the Module 05 acceptance criteria before changing the code. The starter still uses the CSV filename and positional list elements from Assignment 04. I changed FILE_NAME to Enrollments.json, imported the json module, and replaced each student row with a dictionary. The outer students collection remains a list, but each element now has named keys instead of numeric positions. This follows the Module 05 notes on dictionary collections (pp. 2–4).

Defining Constants and Variables

The MENU and FILE_NAME constants are defined once and are not changed while the program runs. The variables for student input and menu selection begin as empty strings, student_data begins as an empty dictionary, and students begins as an empty list. This preserves the organization and readable variable names used in Assignment 04.

```
import json

FILE_NAME: str = "Enrollments.json"

student_first_name: str = ""

student_last_name: str = ""

course_name: str = ""

menu_choice: str = ""

student_data: dict = {}

students: list = []

file = None
```

Changing a List Row to a Dictionary

In Assignment 04, a student record was stored as [first_name, last_name, course_name], and its values were retrieved using indexes such as student[0]. Module 05 explains that a dictionary stores key-value pairs, so values can be accessed using descriptive keys. I use FirstName, LastName, and CourseName to match the supplied JSON file. Dictionary keys are case-sensitive (Module 05 Notes, p. 3).

```
student_data = {
   "FirstName": student_first_name,
   "LastName": student_last_name,
   "CourseName": course_name
}
students.append(student_data)
```

Reading Existing JSON Registrations

At startup, the program opens Enrollments.json in read mode and uses json.load() to turn the JSON array into a Python list of dictionaries. The supplied file already contains Bob Smith and Sue Jones, both enrolled in Python 100. Unlike the CSV approach from Assignment 04, I do not need to split each line at commas. I added try-except-finally handling for missing files, invalid JSON, and file access errors, and close the file in the finally block. I also check that the loaded structure contains dictionaries with the required keys.

```
try:
    file = open(FILE_NAME, "r", encoding="utf-8")
    students = json.load(file)
except FileNotFoundError:
    students = []
except (OSError, json.JSONDecodeError) as error:
    print(f"Unable to load enrollment data: {error}")
    students = []
finally:
    if file is not None and not file.closed:
        file.close()
```

The excerpt above shows the main file-handling pattern; the complete Assignment05.py script also validates the loaded list and dictionary keys.

Registering Students and Handling Input Errors

The while True loop and if/elif/else menu structure remain from Assignment 04. Option 1 prompts for the first name, last name, and course. I used strip() to remove accidental leading or trailing spaces and added try-except blocks to handle blank first and last names. A separate check handles an empty course name. The new dictionary is appended only after the fields are accepted. This applies the structured error-handling topic from Module 05 rather than allowing invalid entries to continue.

```
try:
    student_first_name = input(
        "Enter the student's first name: "
    ).strip()
    if not student_first_name:
        raise ValueError("First name cannot be blank.")
except (ValueError, EOFError) as error:
    print(f"Invalid first name: {error}")
    continue
```

Displaying Current Registrations

Option 2 uses a for loop to display each registration. The f-string accesses the three dictionary keys instead of the numeric indexes used in Assignment 04. I kept long print statements on multiple lines for readability, following the feedback received on earlier assignments.

```
for student in students:
    print(f'Student {student["FirstName"]} '
          f'{student["LastName"]} is enrolled in '
          f'{student["CourseName"]}')
```

Saving Registrations to a JSON File

Option 3 opens Enrollments.json in write mode and uses json.dump() to serialize the students list into JSON. The indent=4 argument makes the saved file easier to read in a text editor. A try-except-finally block reports write errors and closes the file. The program then displays the registrations that were saved. This follows the Module 05 discussion of JSON files and the work-in-memory pattern introduced in Module 04.

```
try:
    file = open(FILE_NAME, "w", encoding="utf-8")
    json.dump(students, file, indent=4)
except (OSError, TypeError) as error:
    print(f"Unable to save enrollment data: {error}")
finally:
    if file is not None and not file.closed:
        file.close()
```

Testing and Results

I checked the program with the supplied starting JSON records and a sample registration for Vic Vu in Python 101. The test sequence was to display existing data, register Vic, display all registrations, save, and exit. After saving, I reopened the JSON data to confirm that the records could be loaded again. The following excerpt illustrates the expected console display:

```
Student Bob Smith is enrolled in Python 100
Student Sue Jones is enrolled in Python 100
Student Vic Vu is enrolled in Python 101
The following data was saved to file!
Program Ended
```

I also checked invalid menu choices, blank first and last names, and the case where the JSON file is missing. The program should still be run in the local PyCharm environment before final submission, as required by the assignment.

Source Control and GitHub

Module 05 introduces GitHub as a location for storing and sharing code. The assignment requires a public repository named IntroToProg-Python-Mod05 containing Assignment05.py, Enrollments.json, and the

knowledge document. After uploading and committing those files, I will place the repository URL here and post the same link to the designated Canvas discussion board.

GitHub repository URL: https://github.com/tdryden11/IntroToProg-Python-Mod05

Conclusion

Assignment 05 builds directly on the collection and file-handling concepts from Assignment 04. Moving from a list of lists to a list of dictionaries makes each registration easier to interpret through named keys. Using json.load() and json.dump() simplifies saving structured data, while try-except-finally blocks make file operations and user input more reliable. These skills will be useful when working with larger data sets and programs that need to recover from errors.

References

Introduction to Programming with Python. (n.d.). Module 05 – Advanced Collections and Error Handling [Course notes].

Introduction to Programming with Python. (n.d.). Module 05 – Advanced Collections and Error Handling [Assignment instructions].

Introduction to Programming with Python. (n.d.). Module 04 – Collections of Data [Course notes].

RRoot. (2030). Assignment05-Starter.py [Course starter script].